

Constructor University Bremen
Fall Semester 2025

Robotics and Intelligent Systems Lab
Michael Görner

Assignment 01 – ROS Introduction

Ali Ahmed Mohamed Amaal Sayed Labib Ibrahim
Yassein Ahmed Mahmoud
Yassin Baher Youssef

Date of Execution: September 19th, 2025

Introduction

This report communicates the results and step-by-step execution of an introductory experiment exploring the basics of ROS workspaces, packages, nodes, and C++ enabled open loop velocity control of a simulated robot through the classic, lightweight ROS turtlesim simulator. While turtlesim is a minimal environment, it still reflects the same publish-subscribe communication architecture that enables distributed processes to operate cooperatively in advanced robotics systems. At its core, the ROS communication framework is built of nodes, independent executables, that exchange structured messages over named topics. The aforementioned abstraction layer decouples information senders from receivers; meaning publishers generate data without explicit knowledge of their subscribers, and subscribers receive data streams without direct references to their sources. This publish-subscribe enabled decoupling is essential for robotics, where the interoperability of different modules, such as sensors, controllers, and actuators, is key for seamless operation.

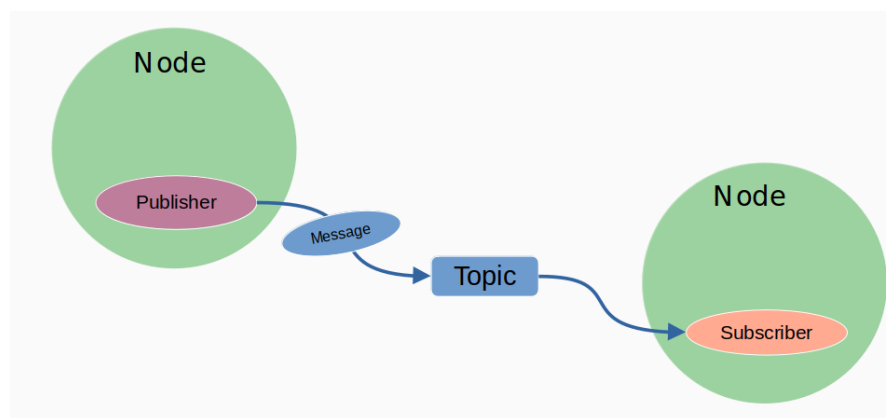


Figure 1: ROS Communication Protocol diagram

(<https://www.roboticsunveiled.com/ros2-ipc-dds-topics-services-actions-interfaces/>)

In ROS, a node is simply a program that is initialized with *roslaunch* and does one job; for example, one node simulates a robot inside of a launched GUI, one node listens to a joystick, one node computes velocity commands, but there always has to be one “master” node where the *roscore* command is ran. They are compiled and run as independent executables (we had to deal with a lot of open terminals...), however this modularity is key as it prevents one failure from crashing the whole system and additionally allows for distribution across multiple machines.

On the other hand, communication between nodes is done by passing messages, which are structured pieces of data defined as *.msg* files. In this lab for example, *geometry_msgs/twist* contained 6 floating-point numbers: linear *x*, *y*, *z* and angular *x*, *y*, *z*. While *turtlesim/pose* has *x*, *y*, θ , and velocity (*v*) values. These message definitions enforce type safety, where a node publishing *twist* can only connect to a subscriber expecting *twist*.

Topics in ROS act as labeled communication channels, much like a named “mailbox” in the system. Any node can “drop” a message in that mailbox (publish), and any number of nodes can “open” that mailbox (subscribe). This architecture enables many-to-many communication; where one publisher can have many subscribers and many publishers can have many subscribers. In the case of our lab, */turtle1/cmd_vel* is the mailbox for velocity commands.

The components mentioned above are connected through the ROS Master (initialized using *roscore*), acting as the name server. The master “introduces” publishers to subscribers, with this action known as the handshake. After this handshake, the publisher and subscriber communicate directly, peer-to-peer over TCPROS (TCP/IP-based message transport protocol used by ROS). However, it is key to note that the master is not in the data path, it only brokers the connection, this makes message passing efficient and scalable.

This introductory lab was key to understanding the complex, yet harmonious, communication architecture briefly described above. Through the creation of simple publisher nodes that generate velocity commands *geometry_msgs/Twist* and the observation of simulator output *turtlesim/Pose*, we are able to study message flow, publishing rates, and topic subscriptions in detail. Tasks such as implementing trajectory-following nodes, including the Haus des Nikolaus pattern, serve as practical demonstrations of how message-passing abstractions translate into observable robotic behavior.

1 Initial Exploration

While running the three nodes (processes): *roscore* (master node), *turtlesim_node*, *turtle_teleop_key*, the following nodes are active:

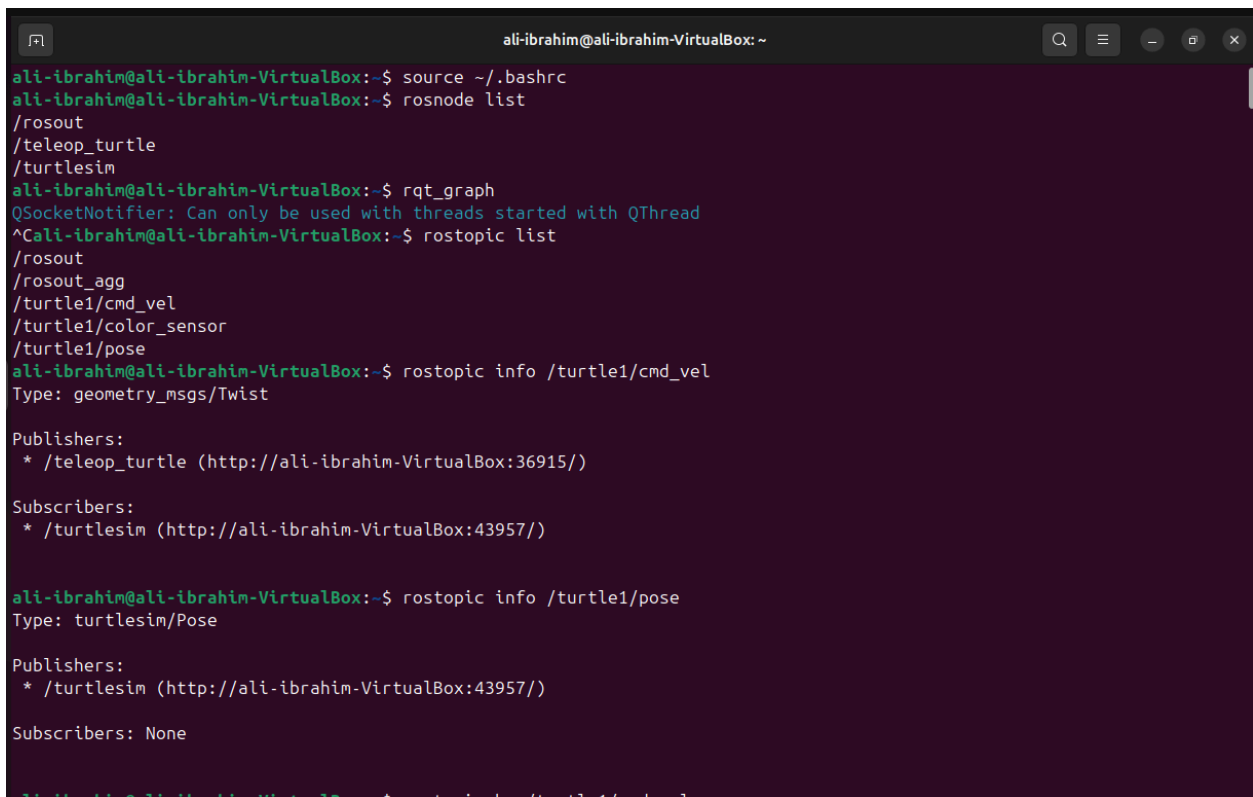
- ***/rosout***
 - Purpose: System logging node automatically started with ROS
 - Publishes: Diagnostic logs to */rosout*
- ***/turtlesim***
 - Publishes:
 - */turtle1/pose* (*turtlesim/Pose*) – The turtle’s x, y, θ linear and angular velocities.
 - */rosout* – Logging messages
 - Subscribes:
 - */turtle1/cmd_vel* (*geometry_msgs/Twist*) – Velocity commands

- ***/turtle_teleop_key***

- Publishes:
 - */turtle1/cmd_vel* (geometry_msgs/Twist) – Velocity commands derived from keyboard input.
 - */rosout* – Logging messages
- Subscribes: None, just listens to keyboard locally.

- **High-level communication overview:**

- */turtle_teleop_key* – **publishes** commands to */turtle1/cmd_vel*.
- */turtlesim* – **listens** to */turtle1/cmd_vel* and **publishes** state on */turtle1/pose*.
- */rosout* – **collects** logs.



```
ali-ibrahim@ali-ibrahim-VirtualBox:~$ source ~/.bashrc
ali-ibrahim@ali-ibrahim-VirtualBox:~$ rosnode list
/rosout
/teleop_turtle
/turtlesim
ali-ibrahim@ali-ibrahim-VirtualBox:~$ rqt_graph
QSocketNotifier: Can only be used with threads started with QThread
^Cali-ibrahim@ali-ibrahim-VirtualBox:~$ rostopic list
/rosout
/rosout_agg
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
ali-ibrahim@ali-ibrahim-VirtualBox:~$ rostopic info /turtle1/cmd_vel
Type: geometry_msgs/Twist

Publishers:
 * /teleop_turtle (http://ali-ibrahim-VirtualBox:36915/)

Subscribers:
 * /turtlesim (http://ali-ibrahim-VirtualBox:43957/)

ali-ibrahim@ali-ibrahim-VirtualBox:~$ rostopic info /turtle1/pose
Type: turtlesim/Pose

Publishers:
 * /turtlesim (http://ali-ibrahim-VirtualBox:43957/)

Subscribers: None

ali-ibrahim@ali-ibrahim-VirtualBox:~$ rostopic info /turtle1/cmd_vel
```

Figure 2: Publisher-subscriber relationships via *rostopic info* command.

As mentioned, messages are also published over topics (or labeled communication channels, informally), their relationships can be explored below:

- ***/turtle1/cmd_vel***

- Published by: */turtle_teleop_key*
- Type: *geometry_msgs/Twist*

- Published only when a key is pressed. No continuous stream unless keys are held down.
- */turtle1/pose*
 - Published by: */turtlesim*
 - Type: *turtlesim/Pose*
 - Published continuously, typically at ~60 Hz.

```
ali-ibrahim@ali-ibrahim-VirtualBox:~$ rostopic hz /turtle1/pose
subscribed to [/turtle1/pose]
average rate: 62.458
  min: 0.014s max: 0.018s std dev: 0.00104s window: 63
average rate: 62.497
  min: 0.012s max: 0.020s std dev: 0.00119s window: 125
average rate: 62.524
  min: 0.012s max: 0.020s std dev: 0.00136s window: 188
average rate: 62.504
  min: 0.012s max: 0.020s std dev: 0.00137s window: 251
average rate: 62.517
  min: 0.012s max: 0.020s std dev: 0.00140s window: 313
average rate: 62.409
  min: 0.012s max: 0.023s std dev: 0.00143s window: 376
average rate: 62.504
  min: 0.007s max: 0.023s std dev: 0.00146s window: 439
average rate: 62.498
  min: 0.007s max: 0.023s std dev: 0.00144s window: 501
average rate: 62.502
  min: 0.007s max: 0.023s std dev: 0.00143s window: 564
average rate: 62.498
  min: 0.007s max: 0.023s std dev: 0.00144s window: 626
average rate: 62.503
  min: 0.007s max: 0.023s std dev: 0.00143s window: 689
average rate: 62.504
  min: 0.007s max: 0.023s std dev: 0.00141s window: 752
average rate: 62.506
  min: 0.007s max: 0.023s std dev: 0.00143s window: 814
```

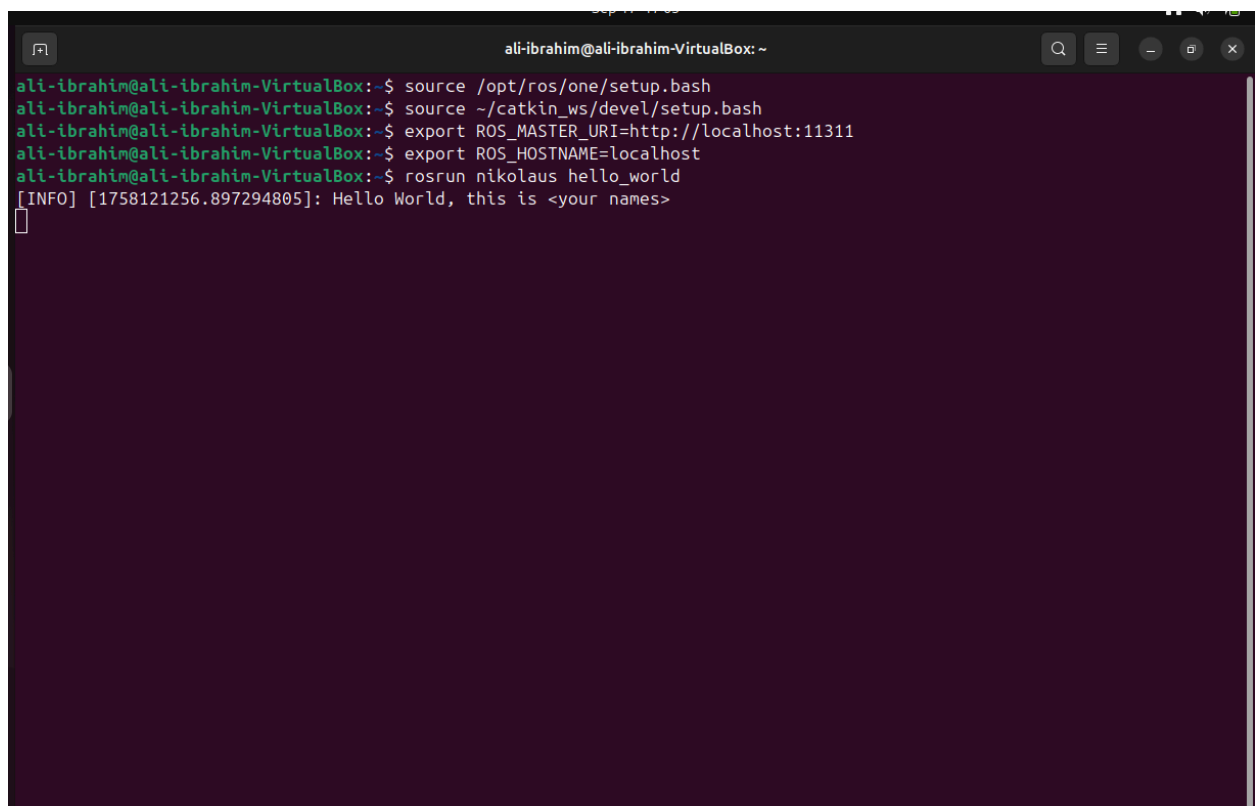
Figure 3: */turtle1/pose* message logs

```
ali-ibrahim@ali-ibrahim-VirtualBox: ~  
min: 0.003s max: 36.630s std dev: 3.51919s window: 108  
average rate: 2.313  
min: 0.003s max: 36.630s std dev: 3.48685s window: 110  
average rate: 2.315  
min: 0.003s max: 36.630s std dev: 3.45530s window: 112  
average rate: 2.282  
min: 0.003s max: 36.630s std dev: 3.44050s window: 113  
average rate: 2.261  
min: 0.003s max: 36.630s std dev: 3.42552s window: 114  
average rate: 2.255  
min: 0.003s max: 36.630s std dev: 3.39562s window: 116  
average rate: 2.234  
min: 0.003s max: 36.630s std dev: 3.36663s window: 118  
average rate: 2.229  
min: 0.003s max: 36.630s std dev: 3.35235s window: 119  
average rate: 2.210  
min: 0.003s max: 36.630s std dev: 3.32444s window: 121  
average rate: 2.201  
min: 0.003s max: 36.630s std dev: 3.31075s window: 122  
average rate: 2.196  
min: 0.003s max: 36.630s std dev: 3.28374s window: 124  
average rate: 2.189  
min: 0.003s max: 36.630s std dev: 3.25738s window: 126  
no new messages  
average rate: 2.143  
min: 0.003s max: 36.630s std dev: 3.24635s window: 127  
no new messages  
average rate: 2.096  
min: 0.003s max: 36.630s std dev: 3.23566s window: 128  
no new messages  
no new messages  
average rate: 1.989  
min: 0.003s max: 36.630s std dev: 3.22588s window: 129
```

Figure 4: `/turtle1/cmd_vel` message logs (apologies for not capturing the “subscribed to: “ line)

2.1 Hello World

Before implementing more advanced control structures, it is essential to verify that the ROS development environment is configured correctly and that the fundamental build pipeline is functional. The “Hello World” node serves this purpose: it introduces the workflow of creating a catkin package, defining dependencies, and compiling a simple C++ executable. Although the program itself performs no robotic control, it demonstrates the lifecycle of a ROS node: initialization, logging, and continuous operation via `ros::spin()`. Successfully completing this task establishes a foundation for the subsequent experiments, ensuring that communication between source code, the build system, and the runtime environment is seamless.

A terminal window titled 'ali-ibrahim@ali-ibrahim-VirtualBox: ~' with a dark background and light green text. The terminal shows the following commands and output:

```
ali-ibrahim@ali-ibrahim-VirtualBox:~$ source /opt/ros/one/setup.bash
ali-ibrahim@ali-ibrahim-VirtualBox:~$ source ~/catkin_ws/devel/setup.bash
ali-ibrahim@ali-ibrahim-VirtualBox:~$ export ROS_MASTER_URI=http://localhost:11311
ali-ibrahim@ali-ibrahim-VirtualBox:~$ export ROS_HOSTNAME=localhost
ali-ibrahim@ali-ibrahim-VirtualBox:~$ rosrn nikolaus hello_world
[INFO] [1758121256.897294805]: Hello World, this is <your names>
```

Figure 5: Hello World terminal command and command line output (apologies for the “<your names>”, it was kept as a generic template).

Running the node and observing the logged message confirmed that the package was correctly built, the dependencies were properly resolved, and the ROS infrastructure was able to execute custom code, thereby validating the development environment for subsequent tasks.

2.2 Move Around

Having confirmed that the ROS development environment is functional with the Hello World example, the next step is to implement active communication between a custom node and the simulator. In this task, the goal is to control the turtle's motion in turtlesim by publishing velocity commands of type `geometry_msgs/Twist` to the topic `/turtle1/cmd_vel`. Unlike the previous task, this program directly influences the state of the simulated robot, thereby demonstrating the principle of publish–subscribe communication in ROS where one node generates motion commands and another node executes them.

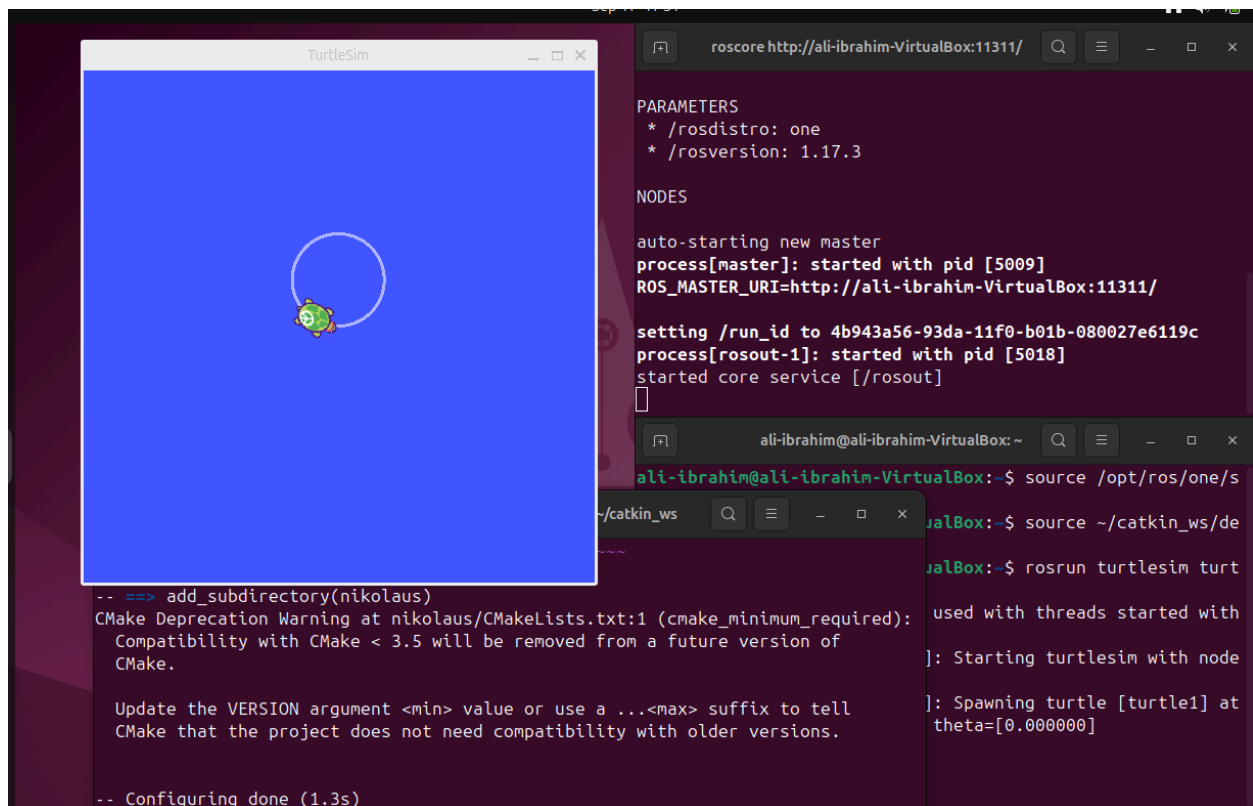


Figure 6: Execution of *circle.cpp*

To produce a figure-eight trajectory, the velocity commands must alternate between leftward and rightward curvature. This can be achieved by periodically inverting the sign of the angular velocity while maintaining a constant linear velocity. In practical terms, the node would alternate between publishing a positive angular velocity (counterclockwise circle) and a negative angular velocity (clockwise circle), each for a fixed duration, resulting in a continuous figure-eight path.

Figure-8 Code Snippet:

```
ros::Rate loop(30);    // smoother control
const double V = 1.0;  // forward speed
const double A = 1.2;  // amplitude of turn rate
const double F = 0.08; // frequency (Hz) of direction change

ros::Time t0 = ros::Time::now();
while (ros::ok()) {
    double t = (ros::Time::now() - t0).toSec();

    geometry_msgs::Twist cmd;
    cmd.linear.x = V;
    cmd.angular.z = A * std::sin(2.0 * M_PI * F * t);
    pub.publish(cmd);

    ros::spinOnce();
    loop.sleep();
}
// Apologies for the code snippet inside of the report, just thought it would be cool to show you we can actually
implement the figure-8 part :)
```

Executing the circle node verified that our code could not only run as a ROS node but also successfully interface with the simulator by producing a sustained motion pattern, thereby reinforcing the importance of correctly publishing structured messages at a fixed rate.

2.3 Haus des Nikolaus

With message publishing verified in 2.2, we now transition from continuous curvature control to piecewise motion planning; a sequence of straight segments and discrete rotations that trace a prescribed geometric pattern. The “Haus des Nikolaus” is an ideal testbed because it demands reproducible line segments, sharp corners, and consistent scaling—properties that expose how timing, message rate, and controller design (open-loop vs. feedback) impact trajectory accuracy in ROS.

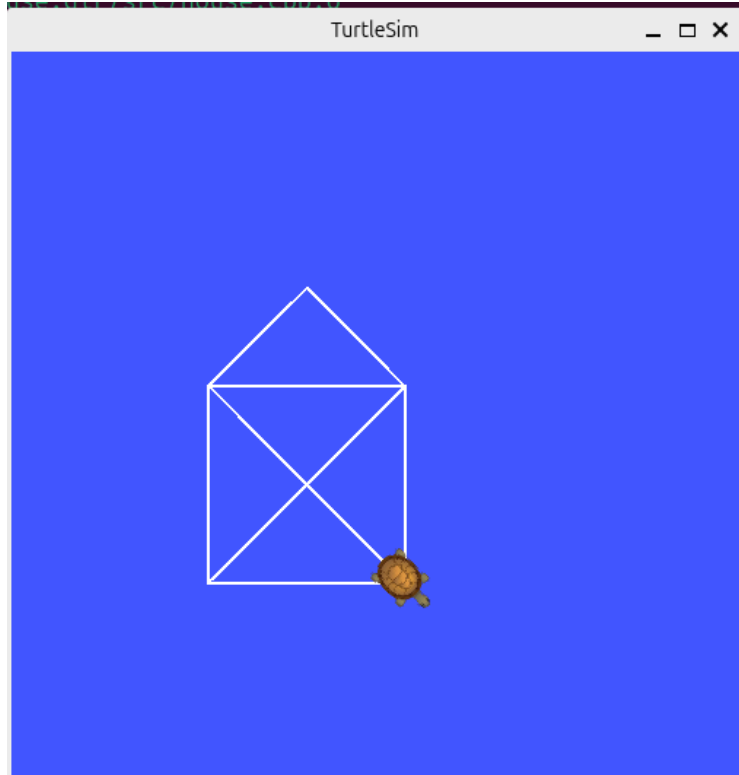


Figure 7: Execution of *house.cpp*

Implemented Logic (what the node does)

1. Abstractions and I/O
 - Publisher: *geometry_msgs/Twist* $\rightarrow$ */turtle1/cmd_vel* (executes motion).
 - Subscriber: *turtlesim/Pose* $\rightarrow$ */turtle1/pose* (estimates state).
2. Path Specification
 - Define vertices on the turtlesim canvas:
 - Square base corners A, B, C, D and roof peak E.
 - Target edge list (straight segments):
 - $A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A$ (square)
 - $A \rightarrow C, C \rightarrow E, E \rightarrow D, D \rightarrow B$ (diagonals + roof)
3. Motion Primitive
 - Rotate-then-Translate: For each segment, first rotate in place to the required heading; then drive straight while holding heading.
4. Control loops:
 - Feedback: subscribe to */turtle1/pose*
 - Rotate using proportional heading control to a tolerance (e.g., $|e_\theta| < 0.01$ rad).

- Then translate with strong heading hold (suppress forward speed if heading error exceeds a threshold) until the Euclidean distance to the waypoint is below tolerance (e.g., $d < 1$ cm (sim units)).

Alternative Approaches & Design Trade-offs

- Different sizes (scaling)
 - Multiply all vertex coordinates by a scalar s around a chosen origin
 - Feedback method: only the waypoint set changes; controller gains remain constant.
- Faster drawing
 - Increase both linear and angular speed limits proportionally to keep corner geometry intact.
- More precise geometry
 - Tighten tolerances, increase control rate (e.g., 100 Hz), and cap angular error before allowing forward motion to avoid rounded corners.
- User-specified shapes
 - Generalize the node to accept a polyline/graph of waypoints from parameters (YAML or ROS params).
 - The same rotate–translate primitive should render arbitrary polygons, stars, or letters with consistent corner quality.

Conclusion

This laboratory introduced us to the essential components of the Robot Operating System and provided hands-on practice in applying its communication architecture to concrete tasks. Beginning with environment setup and a simple “Hello World” node, we validated that our build process and runtime environment were functioning correctly. Extending this foundation, we implemented publisher nodes to control motion in the turtlesim simulator, first with continuous circular trajectories and then with more structured paths such as the figure eight. The final task, drawing the Haus des Nikolaus, required composing motion primitives into a reproducible sequence of straight segments and rotations, which highlighted the distinction between open-loop timing and pose-feedback control.

Through these exercises we not only learned how to create and run ROS nodes, but also observed how topics, messages, and the master node coordinate distributed processes in practice. More broadly, the lab demonstrated how relatively simple message-passing interactions can scale to generate complex and interpretable robot behaviors. This experience provides a foundation for approaching more advanced robotic systems with a clearer understanding of modularity, reproducibility, and the role of communication in real-time control.

Contributions

- Ali Ibrahim
 - 2.3 – Haus des Nikolaus (Complete Task)
 - Wrote house.cpp, ran nodes, validated and tested
 - Final Lab Report

- Yassein Ahmed Mahmoud
 - 2.1 – Hello World (Complete Task)
 - Wrote hello_world.cpp, ran nodes, validated and tested
 - 2.2 – circle.cpp (Complete Task)
 - Wrote circle.cpp, ran nodes, validated and tested

- Yassin Baher Youssef
 - 1 – Initial exploration task
 - Initialized ROS environment, ran turtlesim nodes, executed initial tests

References

- <https://www.roboticsunveiled.com/ros2-ipc-dds-topics-services-actions-interfaces/>
- <http://wiki.ros.org/turtlesim>
- <http://wiki.ros.org/roscpp>
- <http://wiki.ros.org/>